

```

#include <stdio.h>
#include <stdlib.h>
#include <termios.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdbool.h>

// Constante
#define TAILLE 12 // 12 par 12
#define TAILLE_DEPLACEMENTS 999

// a faire:
// virer les sequences inutiles = si dans dep le sokoban fais une "boucle" c'est a dire qu'il revien a une
position ulterieur
// mettre une fonction undo
// affiche le temps
// modifier sauvegarde pour que ca enregistre le fichier dep

// Types
typedef char t_tableau[TAILLE][TAILLE];
typedef char typeDeplacements[TAILLE_DEPLACEMENTS];

// Variables globales
int pos_x_joueur = 0;
int pos_y_joueur = 0;
int compteur = 0;

// Variables pour condition de victoire
int nbCibles = 0;
int placementCorrect = 0;

// prototypes
void charger_partie(t_tableau plateau, char fichier[]);
void enregistrer_partie(t_tableau plateau, char fichier[]);
void afficher_plateau(t_tableau plateau);
void deplacer(t_tableau plateau, t_tableau copie, char touche);
int nombre_cibles(t_tableau plateau);
bool gagne(int nbcible, int placementbon);
void perso_sur_cibles(t_tableau plateau, t_tableau copie);
bool recommencer_partie(char plateau[12][12], char copie[12][12], char fichier[]);
void chargerDeplacements(typeDeplacements t, char fichier[], int *nb);
void enregistrer_dep(typeDeplacements dep, char fichier[], int tailleDep);
int optimiser_fic_dep(t_tableau plateauOriginal, t_tableau plateauCopie, int nbDeplacementsFic,
typeDeplacements t, typeDeplacements *t_copie);
char convertir_mv_touche(char cardep);

```

```

int main() {

    t_tableau plateau;
    t_tableau copie;
    typeDeplacements t;
    typeDeplacements t_copy;

    bool partiegagne = false;
    int nbdeplacementsFic;
    int i = 0;
    char touche;
    int tailleDepOpti;

    char fichier[100];
    char déplacements[100];

    printf("Veuillez saisir le nom d'un fichier : ");
    scanf("%20s", fichier);

    printf("Veuillez saisir le nom du fichier de déplacements : ");
    scanf("%20s", déplacements);

    charger_partie(plateau, fichier);
    chargerDeplacements(t, déplacements, &nbdeplacementsFic);

    tailleDepOpti = optimiser_fic_dep(plateau, copie, nbdeplacementsFic, t, &t_copy );

    for (int i = 0; i < TAILLE; i++)
        for (int j = 0; j < TAILLE; j++)
            copie[i][j] = plateau[i][j];

    nbCibles = nombre_cibles(copie);
    placementCorrect = 0;

    for (int i = 0; i < TAILLE; i++)
        for (int j = 0; j < TAILLE; j++)
            if (plateau[i][j] == '*')
                placementCorrect++;

    afficher_plateau(plateau);
    compteur = 0;

```

```

while (!partiegagne && i < nbdeplacementsFic) {

    touche = convertir_mv_touche(t[i]);
    usleep(250000);

    if (touche != '\0') {

        deplacer(plateau, copie, touche);

    }
    perso_sur_cibles(plateau, copie);
    afficher_plateau(plateau);

    if (gagne(nbCibles, placementCorrect)) {

        char fichierDepSauvegarde[100] = "deplacements_sauvegarde.dep";
        enregistrer_dep(t_copy, fichierDepSauvegarde, tailleDepOpti);

        printf("La suite de déplacements %s est une solution pour la partie %s.\n\n",
            fichier, deplacements);
        printf("Elle contient %d déplacements\n", nbdeplacementsFic);
        printf("Nombre total de déplacements : %d\n", compteur);
        printf("Après optimisation : %d\n", tailleDepOpti);
        printf("Position dans le fichier de déplacements : %d\n\n", i + 1);
        printf("Votre fichier de déplacements a été sauvegardé dans : %s\n", fichierDepSauvegarde);

        partiegagne = true;
    }
    i++;

}

if (!partiegagne) {
    printf("La suite de déplacements %s n'est pas \n une solution pour la partie %s.\n",
        fichier, deplacements);
}

return 0;
}

char convertir_mv_touche(char cardep) {

    if (cardep == 'h' || cardep == 'H') return 'z';
    if (cardep == 'b' || cardep == 'B') return 's';
    if (cardep == 'g' || cardep == 'G') return 'q';

```

```

    if (cardep == 'd' || cardep == 'D') return 'd';

    return cardep;
}

void afficher_plateau(t_tableau plateau) {

    system("clear");

    for (int i = 0; i < TAILLE; i++) {
        for (int j = 0; j < TAILLE; j++)
            printf("%c", plateau[i][j]);
        printf("\n");
    }

    printf("\nDéplacements effectués : %d\n", compteur);
    printf("Cibles totales : %d\n", nbCibles);
    printf("Caisses placées : %d\n", placementCorrect);
}

void charger_partie(t_tableau plateau, char fichier[]) {

    FILE *f;
    char finDeLigne;

    f = fopen(fichier, "r");
    if (f == NULL) {
        printf("ERREUR SUR FICHIER");
        exit(EXIT_FAILURE);
    }

    for (int ligne = 0; ligne < TAILLE; ligne++) {
        for (int colonne = 0; colonne < TAILLE; colonne++) {
            fread(&plateau[ligne][colonne], sizeof(char), 1, f);
            if (plateau[ligne][colonne] == '@') {
                pos_x_joueur = colonne;
                pos_y_joueur = ligne;
            }
        }
        fread(&finDeLigne, sizeof(char), 1, f);
    }

    fclose(f);
}

void enregistrer_partie(t_tableau plateau, char fichier[]) {

    FILE *f;

```

```

char finDeLigne = '\n';

f = fopen(fichier, "w");

for (int ligne = 0; ligne < TAILLE; ligne++) {
    for (int colonne = 0; colonne < TAILLE; colonne++)
        fwrite(&plateau[ligne][colonne], sizeof(char), 1, f);
    fwrite(&finDeLigne, sizeof(char), 1, f);
}

fclose(f);
}

void deplacer(t_tableau plateau, t_tableau copie, char touche) {

    int deltaX = 0, deltaY = 0;
    int nouvelleX, nouvelleY, doubleX, doubleY;
    char caseDevant, caseDerriere;
    bool joueurEtaitSurCible;

    if (touche == 'z' || touche == 'Z') deltaY = -1;
    else if (touche == 's' || touche == 'S') deltaY = 1;
    else if (touche == 'q' || touche == 'Q') deltaX = -1;
    else if (touche == 'd' || touche == 'D') deltaX = 1;
    else return;

    nouvelleX = pos_x_joueur + deltaX;
    nouvelleY = pos_y_joueur + deltaY;
    doubleX = pos_x_joueur + 2 * deltaX;
    doubleY = pos_y_joueur + 2 * deltaY;

    if (nouvelleX < 0 || nouvelleX >= TAILLE || nouvelleY < 0 || nouvelleY >= TAILLE)
        return;

    caseDevant = plateau[nouvelleY][nouvelleX];

    if (doubleX >= 0 && doubleX < TAILLE && doubleY >= 0 && doubleY < TAILLE)
        caseDerriere = plateau[doubleY][doubleX];
    else
        caseDerriere = '#';

    joueurEtaitSurCible = (copie[pos_y_joueur][pos_x_joueur] == '.');

    if (caseDevant == '.' || caseDevant == '#') {

        plateau[pos_y_joueur][pos_x_joueur] = joueurEtaitSurCible ? '.' : '#';
        plateau[nouvelleY][nouvelleX] = (copie[nouvelleY][nouvelleX] == '.') ? '+' : '@';
        pos_x_joueur = nouvelleX;
    }
}

```

```

    pos_y_joueur = nouvelleY;
    compteur++;
}

else if ((caseDevant == '$' || caseDevant == '*') &&
        (caseDerriere == ' ' || caseDerriere == '.')) {

    if (caseDevant == '*') placementCorrect--;

    plateau[pos_y_joueur][pos_x_joueur] = joueurEtaitSurCible ? '.' : ' ';
    plateau[nouvelleY][nouvelleX] = (copie[nouvelleY][nouvelleX] == '.') ? '+' : '@';

    if (copie[doubleY][doubleX] == '.') {
        plateau[doubleY][doubleX] = '*';
        placementCorrect++;
    } else {
        plateau[doubleY][doubleX] = '$';
    }

    pos_x_joueur = nouvelleX;
    pos_y_joueur = nouvelleY;
    compteur++;
}
}

int nombre_cibles(t_tableau plateau) {

    int nb = 0;

    for (int i = 0; i < TAILLE; i++)
        for (int j = 0; j < TAILLE; j++)
            if (plateau[i][j] == '.')
                nb++;

    return nb;
}

bool gagne(int nbcible, int placementbon) {
    return nbcible == placementbon;
}

void perso_sur_cibles(t_tableau plateau, t_tableau copie) {

    for (int i = 0; i < TAILLE; i++)
        for (int j = 0; j < TAILLE; j++)
            if (copie[i][j] == '.' && plateau[i][j] == ' ')
                plateau[i][j] = '.';
}

```

```
void chargerDeplacements(typeDeplacements t, char fichier[], int *nb) {
```

```
    FILE *f;  
    char dep;
```

```
    *nb = 0;  
    f = fopen(fichier, "r");
```

```
    if (f == NULL) {  
        printf("FICHER NON TROUVE\n");  
        return;  
    }
```

```
    while (fread(&dep, sizeof(char), 1, f)) {  
        t[*nb] = dep;  
        (*nb)++;  
    }
```

```
    fclose(f);  
}
```

```
void enregistrer_dep(typeDeplacements dep, char fichier[], int tailleDep) {
```

```
    FILE *f = fopen(fichier, "w");  
    if (!f) {  
        printf("Erreur lors de la sauvegarde des déplacements.\n");  
        return;  
    }
```

```
    for (int i = 0; i < tailleDep; i++)  
        fputc(dep[i], f);
```

```
    fclose(f);  
}
```

```
int optimiser_fic_dep(t_tableau plateauOriginal, t_tableau plateauCopie, int nbDeplacementsFic,  
typeDeplacements t, typeDeplacements *t_copie){
```

```
    int placementbon = 0;  
    int pos = 0;  
    char touche;  
    t_tableau copie;  
    t_tableau avant_coup;  
    int posXdep;
```

```

int posYdep;
int j = 0;
int caseVisite = 0;
int historiqueMvmX[TAILLE_DEPLACEMENTS];
int historiqueMvmY[TAILLE_DEPLACEMENTS];
int historiquePos[TAILLE_DEPLACEMENTS];
int IndiceMvmEnTrop = 0;
bool caisseBouge;

int svg_x = pos_x_joueur;
int svg_y = pos_y_joueur;
int svg_c = compteur;
int svg_p = placementCorrect;

for (int i = 0; i < TAILLE; i++){
    for (int j = 0; j < TAILLE; j++){
        plateauCopie [i][j] = plateauOriginal [i][j];
    }
}

for (int i = 0; i < TAILLE; i++)
    for (int j = 0; j < TAILLE; j++)
        copie[i][j] = plateauCopie[i][j];

for (int i = 0; i < TAILLE; i++){
    for (int j = 0; j < TAILLE; j++){
        if (plateauCopie[i][j] == '*'){
            placementbon++;
        }
    }
}

nbCibles = nombre_cibles(plateauCopie);

while (!gagne(nbCibles, placementCorrect) && pos < nbDeplacementsFic){

    touche = t[pos];

    if (touche != '\0'){
        posXdep = pos_x_joueur;
        posYdep = pos_y_joueur;

        for(int k=0; k<TAILLE; k++)
            for(int l=0; l<TAILLE; l++)
                avant_coup[k][l] = plateauCopie[k][l];

        deplacer(plateauCopie, copie, convertir_mv_touche(touche));
    }
}

```

```

if(pos_x_joueur != posXdep || pos_y_joueur != posYdep){

    caisseBouge = false;
    for(int k=0; k<TAILLE; k++){
        for(int l=0; l<TAILLE; l++){
            if(avant_coup[k][l] != plateauCopie[k][l]){
                caisseBouge = true;
            }
        }
    }
}

if (caisseBouge){
    (*t_copie)[j] = touche;
    j++;
    caseVisite = 0;
}
else{
    IndiceMvmEnTrop = -1;
    for(int i = 0; i < caseVisite; i++){
        if (historiqueMvmX[i] == pos_x_joueur && historiqueMvmY[i] == pos_y_joueur){
            IndiceMvmEnTrop = historiquePos[i];
            caseVisite = i + 1;
            break;
        }
    }
}

if (IndiceMvmEnTrop != -1){
    j = IndiceMvmEnTrop;
}
else{
    if(j < TAILLE_DEPLACEMENTS){
        (*t_copie)[j] = touche;
        historiqueMvmX[caseVisite] = pos_x_joueur;
        historiqueMvmY[caseVisite] = pos_y_joueur;
        historiquePos[caseVisite] = j;
        j++;
        caseVisite++;
    }
}
}
}
pos++;
}

pos_x_joueur = svg_x;
pos_y_joueur = svg_y;
compteur = svg_c;
placementCorrect = svg_p;

```

```
    return j;  
}
```